

大模型面经八股详解

小红书与牛客高频题汇总

版本：2026-07-28 | 适用：大模型算法、LLM 应用、RAG/Agent、训练与推理工程岗位共 160 道题。内容以公开可检索的牛客面经、小红书相关公开索引/二次整理为题目来源，并用原始论文与工程资料校正答案。本文是归纳改写，不复制原帖长段内容。

使用说明

- 1. 第一遍先背每题首段，形成 30-60 秒口述答案；第二遍补公式、工程细节与易错点。
- 2. 项目相关题必须替换成自己的数据规模、模型、指标、失败案例和个人贡献。
- 3. 面试回答建议遵循：结论 → 原理 → 工程权衡 → 项目证据 → 风险/边界。
- 4. 小红书网页存在登录与公开索引限制，因此可直接验证的题目主要来自公开搜索摘要、相关转载和牛客上的“小红书公司面经”；文档已单独标注这一限制。

2026 高频度速览

等级	高频主题	典型追问
S	SFT 数据、LoRA/QLoRA、RAG 全链路、Agent 工具调用	数据怎么构造、如何评测、坏例如何修
S	PPO/DPO/GRPO、DeepSeek-R1	模型数量、损失、组内全对全错、RLVR
A	KV Cache、vLLM、量化、ZeRO	显存估算、prefill/decode、吞吐延迟
A	Planning、Memory、Fallback、多 Agent	循环、超时、权限、评测、稳定性
B	DAPO、GSPO、MCP、动态知识库	与 GRPO/FC 的区别、何时采用
B	多模态结构与评测	视觉 token、projector、图文一致性

目录

- 1. 第一章 Transformer 与注意力机制
- 2. 第二章 Tokenizer、位置编码与现代模型组件
- 3. 第三章预训练、数据工程与 SFT
- 4. 第四章 LoRA、QLoRA、分布式训练与显存估算
- 5. 第五章 RLHF、DPO、PPO、GRPO、DAPO 与 GSPO
- 6. 第六章推理、量化、vLLM 与服务工程
- 7. 第七章 RAG：从文档入库到检索、生成与评测
- 8. 第八章 Agent、工具调用、MCP、记忆与多智能体
- 9. 第九章模型评测、幻觉、安全与 LLM-as-a-Judge
- 10. 第十章多模态大模型基础
- 11. 第十一章项目拷打与系统设计题
- 12. 第十二章常见手撕题、计算题与快速追问
- 13. 附录 A：高频项目追问清单
- 14. 附录 B：资料来源

第一章 Transformer 与注意力机制

本章对应牛客及公开索引面经中最常见的基础追问。面试官通常不会满足于背公式，而会继续问复杂度、掩码、训练与推理差异，以及为什么现代模型做了结构改造。

1. Transformer 的整体结构是什么？

可直接口述：Transformer 由注意力子层、前馈网络、残差连接、归一化和位置编码组成。原始模型是 Encoder-Decoder；现代通用生成式大模型通常采用 Decoder-only，通过因果掩码做自回归预测。

深入展开：输入 token 先映射为向量并加入位置信息。每一层先通过多头注意力让当前位置聚合其他位置的信息，再通过逐 token 的 FFN 做非线性特征变换。残差连接提供稳定的梯度通路，归一化控制激活尺度。Encoder 的自注意力通常可双向观察整段输入；Decoder 的自注意力使用 causal mask，只能看到当前位置之前的 token。原始 Decoder 还包含对 Encoder 输出的交叉注意力。

常见追问：画出张量形状；说明 Pre-Norm 与 Post-Norm；解释训练时并行、生成时串行的原因。

2. Self-Attention 的数学公式和直觉是什么？

给定输入矩阵 $X \in \mathbb{R}^{n \times d}$ ，通过线性映射得到 $Q = XW_Q$ 、 $K = XW_K$ 、 $V = XW_V$ ，注意力为：

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V.$$

直觉上，Query 表示“当前 token 想找什么”，Key 表示“每个 token 能提供什么索引”，二者点积衡量相关性；softmax 将相关性转成权重，再对 Value 做加权求和。注意力不是简单的相似度检索，它的 Q/K/V 都是端到端学习出的任务相关投影。

易错点： QK^T 的形状是 $n \times n$ ；缩放项是 $\sqrt{d_k}$ ，不是 $\sqrt{d_{model}}$ ；softmax 通常沿 Key 维度进行。

3. 为什么点积注意力要除以 $\sqrt{d_k}$ ？

若 Q、K 各维近似独立、均值为 0、方差为 1，则点积的方差随维度增长到约 d_k 。维度较大时，logit 绝对值会很大，softmax 进入饱和区，概率接近 one-hot，梯度变小且训练不稳定。除以 $\sqrt{d_k}$ 后，logit 方差恢复到常数量级。

面试中不要只说“防止梯度消失”，应说明链路：维度增大 $\rightarrow$ 点积方差增大 $\rightarrow$ softmax 饱和 $\rightarrow$ 梯度敏感或过小 $\rightarrow$ 优化困难。

4. Multi-Head Attention 为什么有效？

多头注意力把隐藏维拆成多个子空间，每个头独立计算 Q/K/V 和注意力，使模型能同时学习不同关系，例如局部依赖、指代关系、句法关系、复制模式等。各头输出拼接后再做一次线性映射。

多头不等于“每个头一定有可解释语义”，头之间可能冗余；其主要价值是增加关系建模的多样性和优化自由度。固定总隐藏维时，头数增加会减小单头维度，过多头可能导致每头表达能力不足，并增加 kernel 调度与 KV Cache 元数据开销。

5. Self-Attention 相比 RNN 的核心优势是什么？

第一，训练阶段可对所有位置并行计算，而 RNN 必须按时间步递推。第二，任意两个位置之间的信息路径长度是 $O(1)$ ，更利于学习长距离依赖；RNN 的路径长度是 $O(n)$ 。第三，Transformer 更适合 GPU/TPU 的大规模矩阵乘法和规模化训练。

代价是标准注意力的计算和显存复杂度随序列长度呈 $O(n^2)$ ，而 RNN 每步状态更新通常是 $O(n)$ 。此外，Decoder 自回归推理仍然需要逐 token 生成，所以“Transformer 完全并行”只适用于训练或 prefill，不适用于 decode。

6. 为什么现在的大模型多采用 Decoder-only？

Decoder-only 以统一的 next-token prediction 目标处理生成、问答、代码、对话和上下文学习，训练数据构造简单，架构统一，扩展规律清晰。因果语言建模还天然匹配在线生成。

Encoder-only 更适理解、分类、向量表示；Encoder-Decoder 对输入输出边界明确的任务很强，例如翻译和摘要，但工程上有两套堆栈，参数共享和缓存管理更复杂。Decoder-only 并非所有任务理论上最优，而是在通用生成、规模化预训练、推理服务生态之间取得了较好的综合平衡。

7. Causal Mask 是怎么实现的？训练时为什么仍可并行？

Causal Mask 将注意力矩阵上三角、即“未来位置”的 logit 加上负无穷，softmax 后这些位置权重为 0。训练时整段目标序列已知，可以一次性构造所有位置的 Q/K/V 和掩码，每个位置的计算相互独立，因此可以并行；掩码只限制信息可见性，并不要求按时间步执行。

推理时下一个 token 尚未生成，必须先得到前一个 token，因而 decode 在序列维度上是串行的。KV Cache 只能避免重复计算历史 K/V，不能消除自回归依赖。

8. Cross-Attention 与 Self-Attention 有什么区别？

Self-Attention 的 Q/K/V 来自同一序列；Cross-Attention 的 Q 来自当前解码或查询序列，K/V 来自另一模态或 Encoder 输出。典型例子包括机器翻译 Decoder 读取 Encoder 表示、视觉语言模型的文本 token 读取图像特征。

在张量形状上，若 Query 长度为 n_q 、Key/Value 长度为 n_k ，注意力矩阵为 $n_q \times n_k$ 。Cross-Attention 适合显式的信息桥接，而“把图像 token 和文本 token 直接拼接后做统一自注意力”属于另一种 early-fusion 方案。

9. Attention 的时间和空间复杂度如何分析？

设序列长度为 n 、隐藏维为 d 。Q/K/V 线性投影约为 $O(nd^2)$ ，注意力分数与加权求和约为 $O(n^2d)$ 。当 n 较短、 d 很大时，线性层可能占主要 FLOPs；长上下文时， n^2 项迅速成为瓶颈。

显存方面，朴素实现需要保存 $n \times n$ 注意力矩阵，训练还要保存反向所需中间量。FlashAttention 并没有改变精确注意力的理论 FLOPs 复杂度，而是通过分块、在线 softmax 和重计算减少 HBM 读写与中间矩阵存储，使实际速度和显存

显著改善。

10. Pre-Norm 和 Post-Norm 有何区别？

Post-Norm 是 $\text{LN}(x + \text{Sublayer}(x))$ ，原始 Transformer 使用该形式；Pre-Norm 是 $x + \text{Sublayer}(\text{LN}(x))$ 。Pre-Norm 让残差主干更接近恒等映射，梯度可以沿残差路径稳定传播，因此深层大模型更容易训练，通常对学习率和 warmup 更鲁棒。

Post-Norm 在某些条件下可能获得更强的最终表示，但深层训练更容易不稳定。现代模型常使用 Pre-Norm 或其变体，并搭配 RMSNorm、残差缩放、DeepNorm 等方法。回答时应强调这是优化稳定性和表示质量之间的工程权衡。

11. 为什么 Transformer 使用 LayerNorm，而不是 BatchNorm？

BatchNorm 依赖 batch 维统计量，受 batch size、padding、序列长度和训练/推理模式影响；NLP 中每个 token 的语义和有效长度不同，跨样本统计并不自然。LayerNorm 在单个 token 的隐藏维上归一化，不依赖 batch 大小，训练和推理行为一致，更适合变长序列与分布式训练。

此外，大模型训练常采用很小的 micro-batch、梯度累积和张量并行，BatchNorm 的跨设备同步代价与统计噪声都较高。

12. FFN 在 Transformer 中起什么作用？

注意力主要完成 token 之间的信息混合，FFN 则在每个 token 内对通道进行非线性变换。典型结构为先升维到约 4d，再降回 d；现代模型常用 SwiGLU，形式可写为 $\text{W_down}(\text{SiLU}(\text{W_gate } x)) \odot \text{W_up } x$ 。

FFN 往往占据模型参数和计算的大头，也是 MoE 最常替换的模块。可以把注意力理解为“在哪里取信息”，FFN 理解为“如何加工取到的信息”。两者缺一不可。

13. Attention 权重能否直接解释模型决策？

不能简单等同。注意力权重显示某一层、某一头在一次前向中的加权关系，但模型还有残差、FFN、多层叠加以及 Value 投影；改变注意力权重并不一定按比例改变最终输出。不同参数化还可能得到相似输出但完全不同的注意力图。

因此注意力可作为诊断信号，但不是充分的因果解释。更可靠的方法包括消融、激活干预、梯度归因、因果追踪和 probing。

14. 为什么长上下文下 Attention 会出现“中间遗忘”或注意力稀释？

位置越多，模型需要在更大的候选集合中定位关键信息；训练分布若偏向短文本或关键信息常出现在开头/结尾，模型会形成位置偏置。长上下文还会带来 RoPE 外推误差、检索噪声、softmax 权重稀释和 KV Cache 量化误差。

解决思路包括长上下文继续预训练、位置插值/缩放、检索式上下文管理、层次摘要、稀疏或滑动注意力、关键片段重排，以及针对 needle-in-a-haystack、multi-hop 的专门评测。仅扩大 context window 不等于模型能有效使用全部上下文。

第二章 Tokenizer、位置编码与现代模型组件

这一类题常作为 Transformer 基础后的连续追问，尤其是 RoPE、RMSNorm、SwiGLU、GQA/MQA、MoE 和长上下文。

15. BPE、WordPiece 和 SentencePiece 有什么区别？

BPE 从字符或字节开始，反复合并频率最高的相邻 token 对；WordPiece 也做子词合并，但通常按能提升语言模型似然或某种评分的候选选择；SentencePiece 是一套直接在原始字符串上训练和分词的框架，可实现 BPE 或 Unigram，不依赖预先空格切词。

现代 LLM 常采用 byte-level BPE、SentencePiece BPE 或 Unigram。字节级方案几乎不会出现 OOV，但可能让中文、代码或特殊 Unicode 被切得更碎。面试时要区分“算法”和“工具框架”：SentencePiece 不等于一种唯一合并算法。

16. Tokenizer 为什么会影响模型效果和成本？

Tokenizer 决定同一文本被拆成多少 token，直接影响上下文占用、训练 FLOPs、推理延迟和计费。若某语言被切得更碎，模型需要更多步才能表达相同信息，也更难学习长程结构。代码、数字、JSON 和多语言场景尤其敏感。

词表太小会导致序列变长；词表太大会增加 embedding 与输出层参数、稀疏低频 token 和训练难度。设计时需要在压缩率、跨语言公平性、可逆性、数字处理和特殊 token 需求之间权衡。

17. 绝对位置编码、相对位置编码和 RoPE 有何区别？

绝对位置编码为每个位置加入固定或可学习向量，模型需要自行学习相对关系；相对位置编码直接把位置差引入 attention bias 或打分；RoPE 对 Q/K 的二维子空间做与位置相关的旋转，使点积天然包含相对位置信息。

RoPE 的优势是无额外 attention bias 矩阵、与自回归缓存兼容、相对位置性质好，已广泛用于现代 LLM。缺点是超出训练长度时相位变化可能失真，需要位置插值、NTK-aware scaling、YaRN 等扩展方法。

18. RoPE 的核心推导怎么讲？

将每两个隐藏维看成一个二维向量，在位置 m 处乘旋转矩阵 $R(m\theta)$ 。由于旋转矩阵满足 $R(m)^T R(n) = R(n - m)$ ，所以旋转后的 Query 与 Key 点积只依赖相对位置 $n - m$ ：

$$(R(m)q)^T (R(n)k) = q^T R(n - m)k.$$

不同维度对使用不同频率，低频负责长距离变化，高频负责局部位置。面试不必展开所有复数形式，但应说清“旋转 + 点积将绝对位置转成相对位置差”这一关键。

19. 什么是长度外推，为什么困难？

长度外推是模型在训练上下文长度之外仍保持有效建模能力。困难来自：位置编码在未见位置上的数值分布变化；注意力模式和训练数据长度分布不同；更长序列增加检索和组合难度；KV Cache 量化及数值误差累积。

常见方法包括位置插值、RoPE scaling、长文本继续预训练、稀疏/滑窗注意力、分段记忆和检索增强。要强调：能“跑到”更长长度和在更长长度上“有效推理”是两件事，必须通过长文问答、跨段推理和信息定位评测。

20. RMSNorm 与 LayerNorm 的区别是什么？

LayerNorm 先减均值再除以标准差；RMSNorm 只按均方根缩放，不做中心化：

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\text{mean}(x^2) + \epsilon}} \odot g.$$

RMSNorm 计算略少，实践中可获得接近或更好的稳定性，因而被 LLaMA、Qwen 等模型采用。二者都在隐藏维上操作，不依赖 batch。不能笼统说 RMSNorm 一定更好，它省略了重中心化不变性，效果取决于整体架构与训练配方。

21. GELU、SiLU 和 SwiGLU 有什么区别？

GELU 通过平滑门控保留输入，BERT/GPT 早期常用；SiLU 为 $x \cdot \text{sigmoid}(x)$ ；SwiGLU 使用两路线性投影，其中一路经 SiLU 后与另一路逐元素相乘，再投影回隐藏维。

SwiGLU 通过显式门控提升表达能力，现代 LLM 中十分常见。为了在参数量上与传统 4d FFN 接近，SwiGLU 的中间维通常不会简单取 4d，而会按参数预算调整，例如约 $8d/3$ 并再对齐硬件友好的倍数。

22. MHA、MQA 和 GQA 的区别是什么？

MHA 每个 Query 头都有独立 K/V 头；MQA 让所有 Query 头共享一组 K/V；GQA 将多个 Query 头分组，每组共享一组 K/V。减少 K/V 头数可显著降低 KV Cache 大小和 decode 内存带宽。

MQA 最省缓存，但可能损失质量；GQA 在质量与效率之间折中，成为很多模型的默认选择。注意它主要优化自回归 decode 阶段，prefill 仍有大量矩阵计算。

23. KV Cache 为什么占显存？如何估算？

自回归生成时，每层都要保存历史 token 的 K 和 V。粗略大小可写为：

$$M_{KV} \approx 2LTH_{KV}d_h bB,$$

其中 L 为层数， T 为序列长度， H_{KV} 为 KV 头数， d_h 为单头维度， b 为每元素字节数， B 为并发序列数。

MHA 中 KV 头数等于注意力头数；GQA/MQA 可大幅减少。实际还要考虑 block 对齐、碎片、beam、prefix cache、量化元数据和张量并行分片。KV Cache 随上下文与并发线性增长，是在线服务容量规划的核心。

24. MoE 的基本原理是什么？

Mixture-of-Experts 通常把每层的稠密 FFN 替换成多个专家，路由器为每个 token 选择 top-k 专家，仅激活少量参数。这样总参数量可以很大，但每个 token 的计算量接近较小的稠密模型。

训练难点包括负载不均衡、专家坍塌、跨卡 all-to-all 通信、容量溢出和路由稳定性。常用辅助损失或无辅助负载均衡方法使 token 更均匀地分配。推理时还要区分“总参数量”和“激活参数量”，显存仍需容纳或分布存储全部专家权重。

25. MoE 为什么不等于“只需要加载激活专家”？

在线推理中下一批 token 会路由到不同专家，为保证低延迟，通常需要所有专家常驻某些 GPU 或由 expert parallel 分布在多卡上。若按需从 CPU/磁盘加载专家，传输延迟往往不可接受。

因此 MoE 降低的是每 token FLOPs，不必然同比降低权重显存和通信成本。小 batch 下专家并行利用率也可能较差；大规模服务需要请求聚合、路由均衡和高效 all-to-all。

26. 什么是权重绑定 (weight tying)?

输入 embedding 矩阵与输出语言模型头共享参数，即输出 logits 使用 embedding 的转置。它可减少参数量，并让输入和输出词表示处于同一空间。很多语言模型采用该技巧，但并非所有模型都绑定，尤其在词表很大、并行切分或架构特殊时可能分开。

若面试官问参数估算，要确认 embedding 与 lm_head 是否共享，否则会差一个 $\text{vocab_size} \times \text{hidden_size}$ 。

第三章预训练、数据工程与 SFT

真实面经中，数据构造几乎总会结合项目追问。高质量回答必须覆盖来源、清洗、去重、配比、模板、损失掩码、训练监控和评测闭环。

27. 大模型从预训练到可用对话模型通常经历哪些阶段?

典型流程包括：大规模自回归预训练，学习语言、知识和基础能力；领域继续预训练或中期训练，补充特定领域与长上下文能力；SFT 学习指令遵循、格式和任务行为；偏好优化或强化学习进一步改善有用性、安全性和推理策略；最后经过量化、蒸馏、部署与在线评测。

不同模型不一定严格按此顺序，也可能多轮交替。回答时要将“能力来源”和“行为对齐”分开：预训练决定大部分知识与能力上限，后训练决定模型是否按用户期望稳定使用这些能力。

28. 预训练的 next-token prediction 目标是什么?

给定 token 序列 $x_1, \dots, x_T$ ，最大化条件似然：

$$\sum_{t=1}^T \log p_{\theta}(x_t | x_{<t}),$$

实现中通常最小化 token 级交叉熵。训练时用 teacher forcing，将真实前缀作为输入；推理时模型依赖自身生成的前缀，因此会有 exposure bias。

Loss 下降并不等价于所有能力提升。数据分布、token 权重、难度和评测泄漏都会影响结果，必须配合下游 benchmark 与人工评测。

29. Scaling Law 的面试回答应该包含什么?

Scaling Law 描述模型规模、数据量、计算量与损失之间的经验幂律关系。核心不是“模型越大越好”，而是在给定计算预算下平衡参数量和训练 token 数，避免模型过大但数据不足，或数据很多但模型容量不足。

实际项目还要考虑数据质量、架构、优化器、硬件利用率和推理成本。Chinchilla 风格结论推动了更充分的数据训练；现代模型常采用更长训练、数据混合和多阶段训练。不要把某个固定的“参数:token 比例”当成永恒定律。

30. 如何构建领域继续预训练数据?

先定义领域边界和能力目标，再收集权威文档、业务语料、代码、问答、日志等；进行格式解析、语言识别、去重、质量打分、隐私与安全过滤；按来源、质量、时间和难度分层，设计领域数据与通用数据配比；最后做小规模 ablation 确认收益和遗忘程度。

继续预训练通常使用纯文本 next-token loss，不要求全部转成 QA。若只把少量知识做成问答 SFT，模型可能学会答题风格，却没有充分吸收领域分布。保留一定通用数据可缓解灾难性遗忘。

31. SFT 的核心目标和流程是什么?

SFT 使用高质量示范数据，让模型学习指令遵循、对话格式、任务结构和期望回答。流程包括任务定义、数据收集/合成、清洗去重、模板化、tokenize、response loss mask、训练、离线评测、人工抽检和迭代。

SFT 不是简单“把文档喂给模型”。数据应体现用户输入到理想输出的映射，包括边界条件、拒答、工具调用格式和多轮状态。质量、覆盖度和分布通常比盲目增加数量更重要。

32. SFT 数据常见格式有哪些?

常见单轮格式是 {instruction, input, output}；对话格式是 messages=[{role: system/user/assistant, content: ...}]；工具调用数据还包含 tool schema、assistant tool call 和 tool result。

训练前要用与部署一致的 chat template 展开，并明确哪些 token 计入 loss。通常只对 assistant 回复计算 loss，system/user 作为条件；也可对全部 token 训练，但会浪费容量学习复述提示词，并改变损失权重。多轮数据要避免把后续真实答案泄漏到前文。

33. SFT 数据集如何清洗？

至少包含：结构合法性检查；乱码、模板残留和截断检测；规则与模型联合质量打分；精确去重和语义去重；PII、版权与安全过滤；长度、语言、领域、难度和任务类型分布统计；人工分层抽检。

对代码和数学数据还要执行单元测试、编译或答案验证。对合成数据应检查教师模型的系统性偏差、同质化和措辞重复。清洗不能只在样本级做，还要检查 `prompt` 泄漏、`benchmark` 污染和不同 `split` 的近重复。

34. 为什么要做去重？精确去重和语义去重如何实现？

重复数据会放大特定样本权重，造成记忆、评测污染和数据分布失衡。精确去重可对规范化文本做哈希；近重复可用 MinHash/LSH、n-gram Jaccard、SimHash；语义去重可用 `embedding` 聚类或近邻搜索。

阈值过低会误删模板相似但语义不同的样本，阈值过高又保留大量改写。工程上常分层处理：先文档级精确/近重复，再段落级或问答级语义去重，并保留来源和版本信息以便追踪。

35. SFT 数据量多少合适？

没有脱离任务和质量的固定答案。窄领域、输出格式明确的任务，数千条高质量数据即可看到明显变化；通用聊天或复杂推理需要更大、更多样的数据。应通过学习曲线和边际收益决定，而不是背“几万条”。

具体做法是固定评测集，逐步扩大数据量或增加不同数据桶，观察任务成功率、通用能力、过拟合和遗忘。若训练 `loss` 很低但验证收益停滞，继续堆同质数据意义有限。

36. 如何设计 SFT 数据配比？

先按任务、领域、语言、难度、长度、来源和安全类别划分数据桶。配比应服务于目标能力，而不是按原始数据量直接采样，否则海量低价值类别会淹没稀缺高价值任务。

可采用温度采样、上限截断、难例过采样和 `curriculum`。训练中监控分桶 `loss` 与评测，动态调整。通用数据用于维持基础能力，领域数据强化目标任务，拒答和负面示例用于边界行为。

37. 什么是数据污染？如何检测 benchmark 泄漏？

数据污染指训练数据包含评测题、答案或高度相似改写，导致指标虚高。检测可结合精确匹配、n-gram 重合、MinHash、`embedding` 近邻、关键实体/数字组合和时间戳审计。

对代码题还要查题面、函数签名、测试样例和公开解答；对选择题要考虑打乱选项后的相似性。发现污染后应删除相关训练样本、重建时间切分评测集，或使用私有/动态生成测试。

38. 什么是 response-only loss masking？

将 `user/system/padding` 等 `token` 的 `loss` 权重设为 0，只对 `assistant` 目标 `token` 计算交叉熵。这样优化信号聚焦于模型应生成的内容，尤其适合对话 SFT。

实现时要确保模板边界和 `special token` 定位正确；多轮对话中可对每个 `assistant span` 计算 `loss`。常见 bug 是错误地把 `tool result` 或 `user` 内容纳入标签，或 `shift labels` 后 `mask` 对不齐。

39. SFT 为什么会出现过拟合或灾难性遗忘？

领域数据小且同质时，模型容易记住措辞和格式，通用能力下降；学习率过大、`epoch` 过多或全参微调会使参数偏离预训练分布。评测上表现为训练 `loss` 持续下降，但验证集、开放域问答或安全能力恶化。

缓解方法包括混合通用 `replay` 数据、较小学习率、早停、PEFT、正则化、较少 `epoch`、多任务训练和更严格的数据去重。还应评估“目标提升 / 通用损失”的 Pareto，而非只看单一任务。

40. 合成数据有哪些优点和风险？

优点是低成本扩展覆盖、生成难例、控制格式和补齐长尾；风险是教师错误被放大、风格单一、答案过于模板化、分布与真实用户不一致，以及模型自训练导致模式坍塌。

较可靠流程是：用真实种子和明确 `taxonomy` 驱动生成；多教师或多采样增加多样性；规则、执行器、检索和判别模型做验证；人工抽检高风险类别；记录 `provenance`。合成数据应补充真实数据，而不是无验证地完全替代。

第四章 LoRA、QLoRA、分布式训练与显存估算

面试常从“用过 LoRA 吗”追问到秩、初始化、目标层、显存、合并权重、ZeRO 和并行策略。回答要同时覆盖算法与系统。

41. LoRA 的原理是什么？

LoRA 假设下游适配所需的权重更新具有低内在秩。冻结原权重 W ，学习：

$$W' = W + \Delta W, \quad \Delta W = \frac{\alpha}{r} BA,$$

其中 A 、 B 的秩 r 远小于输入输出维度。训练时只保存 A/B 的参数、梯度和优化器状态，从而降低显存和存储成本。

推理时可把 BA 合并进 W ，不增加额外延迟；也可保留 `adapter` 以便动态切换多个任务。LoRA 降低的是可训练参数相关开销，冻结基座的前向激活和计算仍然存在。

42. LoRA 的 A 、 B 通常如何初始化？为什么？

常见做法是一个矩阵随机初始化、另一个初始化为 0，使训练开始时 $\Delta W = 0$ ，模型初始行为与基座完全一致。例如 A 随机、 B 为 0；具体方向因实现定义而异。

若两者都为 0，初始梯度可能无法同时有效传播；若两者都随机，初始就扰动基座输出。PiSSA、LoftQ 等方法会利用原权重的 SVD 或量化误差做更有信息的初始化，以加快收敛或改善量化适配。

43. LoRA 的 rank、alpha 和 dropout 如何选择？

rank r 控制适配容量与开销；alpha 通常配合 α/r 缩放更新幅度；dropout 对 LoRA 分支做正则。没有通用最优值，常见 r 从 8、16、32 到 64，复杂任务或多层适配可能需要更高。

选择应做分层 ablation：观察验证收益、训练稳定性和参数预算。若 r 增大收益很小，说明瓶颈可能在数据或目标层；若欠拟合，可增加 r 、覆盖更多层或改用全参微调。不要把 alpha 简单理解为学习率，它改变的是 LoRA 更新的尺度。

44. LoRA 应插在哪些层？只能插 Linear 吗？

最常见是在注意力的 $q/k/v/o$ 投影和 FFN 的 `up/down/gate` 线性层。数学上 LoRA 可应用于任何可表示为矩阵乘法的权重，包括卷积展开后的权重、embedding 等；不是“只能 Linear”。

是否适配全部线性层取决于任务。仅 q/v 成本最低但容量有限；全线性层通常效果更强。LayerNorm 的缩放和偏置是一维参数，直接套低秩分解没有意义，若需要可单独解冻 norm 参数，而不是声称“LoRA 不能放在 LayerNorm 后面”。

45. LoRA 推理时必须挂着 Adapter 吗？

不必。对单个固定任务，可将 LoRA 更新合并到基座权重，得到与显式 adapter 基本等价的线性变换，减少运行时分支。若需要一套基座动态服务多个 adapter，则保留 adapter 并在请求级路由。

合并时要注意量化权重、dtype、多个 adapter 叠加和不可逆覆盖。对 4-bit 基座，通常需要先以更高精度反量化/合并再重新量化，直接合并可能带来额外误差。

46. QLoRA 的核心技术是什么？

QLoRA 将基座权重以 4-bit 量化冻结，反向传播穿过量化权重，只训练高精度 LoRA 参数。其关键技术包括适合近似正态权重分布的 NF4、对量化常数再次量化的 double quantization，以及缓解显存峰值的 paged optimizer。

计算通常仍在 BF16/FP16 中完成，4-bit 主要用于存储基座权重。QLoRA 显著降低权重显存，但激活、LoRA 参数、部分临时缓冲和 KV/attention 中间量仍可能导致 OOM。

47. NF4 为什么适合神经网络权重？

许多预训练权重在分组归一化后近似零均值正态分布。NF4 的量化级别按正态分布的分位数设计，使每个量化区间具有近似相等概率，较均匀整数间隔更有效利用有限码点。

它不是说所有权重严格正态，也不保证任何模型上都最优。实际效果还受 group size、异常值、计算 dtype 和校准影响。

48. 全参微调、LoRA 和 QLoRA 如何选择？

全参微调容量最高，适合数据充足、预算大、需要深度改变模型能力的场景，但训练与存储昂贵且遗忘风险大。LoRA 适合多任务、快速迭代和中等资源；QLoRA 进一步降低显存，适合单机或有限 GPU，但量化噪声和 kernel 支持可能影响稳定性与速度。

决策应看：模型规模、目标能力变化幅度、数据量、硬件、是否需要多 adapter、部署 dtype 和可接受的性能损失。不能只按“效果排序”，还要看总拥有成本。

49. 训练显存由哪些部分组成？

主要包括模型权重、梯度、优化器状态、主权重副本、激活、临时通信/算子缓冲和内存碎片。以 Adam 混合精度全参训练为例，每参数可能包含 2 字节 FP16/BF16 权重、2 字节梯度、8 字节一阶/二阶状态，以及可能的 4 字节 FP32 主权重，粗略达到 12-16 字节/参数，具体取决于实现。

激活与 batch、序列长度、层数和隐藏维相关，长序列下可能比参数状态更大。做估算时应明确 dtype、优化器、checkpointing、ZeRO stage 和并行切分。

50. Gradient Checkpointing 为什么省显存？代价是什么？

正常反向需要保存大量前向激活。Checkpointing 只保存部分层边界，在反向时重新计算中间前向，从而用额外计算换显存。通常能显著降低随层数增长的激活存储。

代价是训练时间增加、重计算带来额外随机性管理和 kernel 开销。与 FlashAttention、sequence parallel 等可叠加。它不减少权重、梯度或优化器状态。

51. DeepSpeed ZeRO-1、2、3 的区别是什么？

ZeRO-1 在数据并行 rank 间切分优化器状态；ZeRO-2 进一步切分梯度；ZeRO-3 再切分模型参数，计算某层时按需 all-gather 参数。随着 stage 增加，单卡显存下降，但通信、调度和实现复杂度上升。

ZeRO-3 不等于参数永远只占 1/N：运行时会有预取、all-gather buffer、未释放参数和碎片。CPU/NVMe offload 可进一步省 GPU 显存，但可能受 PCIe、CPU 内存和 I/O 限制。

52. 数据并行、张量并行、流水线并行、专家并行分别是什么？

数据并行复制模型、切分 batch，并同步梯度；张量并行在单层内部切分矩阵，通信频繁但可容纳大层；流水线并行按层切 stage，用 micro-batch 填充流水线，存在 bubble；专家并行把 MoE 专家分布到不同设备，token 通过 all-to-all 路由。

大模型训练通常组合 3D/4D 并行。选择取决于单层能否放入一张卡、网络拓扑、batch 大小、序列长度和模型是否是 MoE。

53. 什么是梯度累积？它与大 batch 完全等价吗？

梯度累积在多个 micro-batch 上累加梯度，再执行一次 optimizer step，从而在显存受限时获得更大的有效 batch：

$$B_{global} = B_{micro} \times N_{accum} \times N_{DP}.$$

在无随机差异、loss 正确归一化且参数在累积期间不更新时，理论上接近大 batch。但 dropout 随机性、BatchNorm、动态 loss scaling、梯度裁剪位置、学习率调度按 step 计数等会造成差异。

54. 为什么混合精度训练常用 BF16 而不是 FP16？

BF16 与 FP32 具有相同的 8 位指数范围，不易溢出/下溢；尾数精度较低但深度学习通常可容忍。FP16 尾数更多、指数范围小，常需 loss scaling 防止梯度下溢。

在支持 BF16 的硬件上，它通常更稳定。某些算子、通信或权重仍会保留 FP32，且优化器状态常用 FP32。混合精度不等于所有张量都是同一种低精度。

第五章 RLHF、DPO、PPO、GRPO、DAPO 与 GSPO

这是 2025-2026 年大模型算法岗最密集的进阶考区。回答时应先讲目标与数据，再讲损失、模型数量、on/off-policy、优势估计、稳定性问题和适用场景。

55. 为什么 SFT 之后还需要偏好对齐或强化学习？

SFT 学习的是示范数据中的条件分布，能让模型“像答案”，但不直接优化多个可行答案之间的相对偏好、长程任务成功率或可验证结果。高质量示范昂贵，也难覆盖所有坏行为。

偏好训练利用 chosen/rejected 差异学习人类偏好；RLVR 则可直接优化数学、代码、工具执行等可验证 outcome。它们能提升帮助性、安全性、推理策略和探索能力，但也可能导致 reward hacking、能力退化或输出过长，因此需要 KL 约束、数据混合与系统评测。

56. 经典 RLHF 的三阶段流程是什么？

第一步用高质量指令数据做 SFT，得到初始策略；第二步收集同一 prompt 下多个回答的人工偏好，训练 Reward Model；第三步用 PPO 等强化学习算法优化策略，使其获得更高奖励，同时通过参考模型的 KL 惩罚限制偏移。

工程中常同时存在 policy、reference、reward、value 四类模型或模块。Reference 和 reward 通常只推理，policy 与 value 需要训练。也可共享 backbone 或使用分布式并行降低成本。

57. Reward Model 如何训练？

输入通常是 (prompt, response)，模型输出标量 reward。对偏好对 (y_w , y_l)，常用 Bradley-Terry / pairwise logistic loss：

$$L = -\log \sigma(r(x, y_w) - r(x, y_l)).$$

它只要求 preferred 回答分数高于 rejected，不直接确定绝对标尺。数据应控制长度、风格等混杂因素，避免 RM 学到“越长越好”或特定模板。评估除 pairwise accuracy 外，还要看跨域泛化、校准、对抗样本和与人工排序的一致性。

58. Reward Model 和 Value Model 有什么区别？

Reward Model 通常对完整回答或状态-动作给出外部奖励，代表任务/偏好目标；Value Model 估计从当前状态继续按策略执行的期望累积回报，用于计算 advantage、降低策略梯度方差。

在 LLM PPO 中，RM 可只在序列末尾给一次分数，Value Model 则通常对每个 token 状态输出 value。两者可能结构相似，但训练目标、输入时机和语义不同。GRPO 的主要动机之一就是用组内相对奖励构造 baseline，省去独立 critic/value model。

59. PPO 的核心思想和损失是什么？

PPO 使用旧策略采样轨迹，计算 advantage，并限制新旧策略变化。经典 clipped objective 为：

$$L^{clip} = E[\min(r_t A_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) A_t)],$$

其中 $r_t = \pi_\theta(a_t|s_t)/\pi_{old}(a_t|s_t)$ 。LLM RLHF 中还加入 value loss、entropy bonus、reference KL penalty。

clip 不是保证绝对不更新，而是抑制会进一步扩大目标的过大概率比率。PPO 相对稳定，但模型多、rollout 贵、超参数敏感且 on-policy 数据利用率低。

60. PPO 中 KL 约束有什么作用？

Reward Model 只近似人类偏好，策略若不受约束会寻找 RM 漏洞，产生 reward hacking。将策略与 reference model 的 KL 作为惩罚，限制输出分布偏离 SFT 模型过远，保留语言质量与通用能力。

KL 系数过小会导致策略漂移、重复和奇怪格式；过大则模型几乎不学习。可使用固定系数或根据实际 KL 与目标 KL 自适应调整。注意 token 级 sampled KL 估计可能有高方差，且实现符号很容易写反。

61. 什么是 GAE？

Generalized Advantage Estimation 用 TD residual 的指数加权和估计 advantage：

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t),$$

$$A_t = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}.$$

λ 在低偏差高方差的 Monte Carlo 回报与高偏差低方差的一步 TD 之间权衡。LLM 中序列通常有限、奖励可能集中在末尾，value 估计质量会显著影响 PPO 稳定性。

62. DPO 的核心思想和损失是什么？

DPO 从带 KL 正则的 RLHF 最优策略形式出发，把隐式 reward 表示成策略相对 reference 的 log-ratio，直接用偏好对训练，无需显式 RM 和在线 RL。常见损失为：

$$L_{DPO} = -\log \sigma(\beta[(\log \pi_\theta(y_w|x) - \log \pi_{ref}(y_w|x)) - (\log \pi_\theta(y_l|x) - \log \pi_{ref}(y_l|x))]).$$

它工程简单、稳定、可离线训练，但依赖偏好数据覆盖和 reference，无法像在线 RL 那样主动探索新的高奖励轨迹。

63. DPO 中 beta 的含义是什么？

beta 控制策略相对 reference 的偏离强度，也影响偏好 margin 的缩放。直觉上 beta 较大时 KL 约束更强、更新更保守；不同实现的参数约定可能不同，必须结合损失定义确认。

调参时观察 chosen/rejected reward margin、隐式 KL、验证胜率、长度和通用能力。只看训练 loss 下降不足以判断 DPO 是否变好。

64. DPO 训练时 chosen 概率一定上升、rejected 概率一定下降吗？

不一定。DPO 优化的是 chosen 相对于 rejected、并相对于 reference 的对数概率差。模型可以通过更大幅度降低 rejected，或在全局归一化约束下同时降低二者但改善相对 margin 来降低损失。

因此应监控 chosen/rejected log-prob、margin、隐式 reward、KL 和生成质量。把 DPO 误解为两个独立的“正样本上升、负样本下降”交叉熵是常见错误。

65. DPO 与 PPO 的主要区别是什么？

DPO 是离线偏好优化，使用固定 chosen/rejected 对，像监督学习一样反向；PPO 是 on-policy RL，需要当前策略 rollout、奖励、advantage 和多模型协作。DPO 成本低、实现简单，适合稳定的偏好数据；PPO 可探索策略当前分布，并直接优化任意奖励，适合可验证任务和在线改进。

DPO 的上限受离线数据支持集限制，且对 reference、beta、偏好噪声和长度偏差敏感；PPO 则更容易训练不稳定、reward hacking 和资源爆炸。

66. DPO 常见失败模式有哪些？

包括偏好对质量差或差异维度过多；长度偏差导致模型偏长/偏短；reference 不合适；beta 设置失衡；chosen/rejected 都是低质量；离线数据与当前策略分布差距过大；过拟合模板；通用能力遗忘。

应构造“最小差异”的偏好对，加入难负样本与长度控制，做去重和分桶评估，监控隐式 KL，并混合 SFT 或通用数据。对多目标偏好可分维度标注或训练多 reward，而不是把所有偏好揉成一个不透明标签。

67. ORPO、KTO、IPO 与 DPO 的关系如何简述？

它们都试图降低传统 RLHF 的工程复杂度。IPO 修改偏好目标以缓解 DPO 在可分数据上 margin 无限增大的倾向；KTO 可利用单条 desirable/undesirable 反馈而不要求严格成对；ORPO 将监督似然与偏好 odds ratio 目标结合，通常无需独立 reference 模型。

面试中不必死背所有公式，应说明它们在数据形式、是否需要 reference、稳定性和目标偏差上的区别，并根据业务可获得的反馈类型选择。

68. 什么是 RLAIIF 和 Constitutional AI？

RLAIIF 用 AI 评审代替或补充人工偏好标注，可快速规模化，但会继承评审模型偏差。Constitutional AI 使用一组原则指导模型自我批评、修订和偏好生成，再做监督或强化学习。

关键不是“完全不要人”，而是把人类投入前移到原则、rubric、校准与审计。实际系统常采用多评审、随机人工复核、位置交换、长度去偏和规则验证降低 judge bias。

69. GRPO 的核心思想是什么？

GRPO 对同一 prompt 采样一组回答，计算各自 reward，并以组内均值/标准差归一化得到相对 advantage，例如：

$$A_i = \frac{r_i - \text{mean}(r_{\text{group}})}{\text{std}(r_{\text{group}}) + \varepsilon}.$$

然后使用类似 PPO 的 importance ratio、clip 和 KL 正则更新策略。它省去独立 value/critic model，降低显存和训练复杂度，特别适合数学、代码等可一次采样多个解并获得可验证奖励的 RLVR。

70. GRPO 与 PPO 的区别是什么？

PPO 通常通过 Value Model 估计每个状态的 baseline，并用 GAE 计算 token 级 advantage；GRPO 用同 prompt 多个回答的组内相对 reward 作为 baseline，不训练 critic。GRPO 更省资源，但需要每 prompt 多采样，rollout 成本仍高。

组内奖励方差为零时 advantage 退化；不同 prompt 难度使组归一化可能改变跨样本权重；序列级奖励广播到 token 也带来信用分配问题。

71. GRPO 中一个 group 全对或全错会发生什么？

若所有回答 reward 相同，标准化后的 advantage 约为 0，该组不给有效策略梯度，浪费 rollout。全对说明题目过易或探索不足；全错说明题目过难、奖励稀疏或策略尚无法覆盖正确解。

解决可用 dynamic sampling 丢弃或减少零方差组，维护合适难度分布；引入过程奖励、部分分；扩大采样多样性；使用 leave-one-out baseline 或课程学习。不能简单给全对组统一正 advantage，否则会失去相对比较并可能放大低质量轨迹。

72. RLVR 是什么？与 RLHF 有何区别？

RLVR 使用可自动验证的奖励，例如数学最终答案、代码测试通过率、定理检查、工具执行成功、格式约束。其信号客观、可规模化，减少人工 RM，但适用范围限于可验证任务。

RLHF 更关注主观帮助性、安全性和风格，需要人类偏好或 learned reward。实际 reasoning 模型常组合 outcome reward、格式奖励、安全/语言奖励与 SFT 数据。自动可验证也不意味着不会 reward hacking，测试覆盖不足时模型可能钻漏洞。

73. DeepSeek-R1-Zero 与 DeepSeek-R1 的训练思路有何区别？

R1-Zero 从 base model 直接做大规模 RL，推理行为可自然涌现，但存在可读性差、语言混杂等问题。R1 在此基础上加入 cold-start SFT 数据、多阶段 reasoning RL、拒绝采样 SFT 和面向通用能力/对齐的后续 RL，使输出更可读、更通用。

面试重点应是：纯 RL 能激励推理策略，但高质量冷启动和多阶段后训练对稳定格式、语言一致性、通用任务与安全仍重要。

74. DAPO 相比朴素 GRPO 做了哪些改进？

DAPO 针对长 CoT RL 的训练问题提出四类关键技巧：Clip-Higher，为正 advantage 提供更高的上裁剪范围以鼓励探索；Dynamic Sampling，过滤全对/全错等无梯度组；Token-level Policy Gradient Loss，按 token 聚合以降低长短样本权重偏差；Overlong Reward Shaping，对截断的超长回答做软惩罚而非粗暴处理。

其价值不只是一个新公式，而是一套可规模化 RL 训练配方，重点解决熵坍塌、无效样本、长度偏差和超长轨迹。

75. 什么是熵坍塌？如何缓解？

策略熵快速下降意味着采样结果趋于单一，探索不足，组内回答高度相似，RL 容易早熟收敛。原因可能是 clip 不对称、奖励过强、temperature 太低、KL/entropy 配置不当或训练数据过易。

缓解包括提高采样温度/多样性、entropy bonus、Clip-Higher、难度课程、动态采样、减小学习率、合理 KL 和混入探索性 prompt。要同时看 token entropy、distinct-n、group reward variance 和验证集，而不是仅看平均 reward。

76. GRPO 的长度偏差从哪里来？

若把序列级 advantage 广播到所有 token，再对全 batch 的 token 求平均，长回答会贡献更多 token 梯度；反之若先对每个序列 token 平均，长短样本权重又可能出现另一种偏差。截断回答和长度相关 reward 进一步加剧问题。

Dr. GRPO、DAPO 等工作关注不同归一化方式。工程上需明确损失分母是 token 数、序列数还是固定最大长度，并按任务控制长度 reward、EOS 与截断处理。

77. GSPO 为什么使用序列级 importance ratio？

GSPO 认为对自回归序列，token 级 ratio 与序列级 reward/采样单位不一致，长序列中噪声和 clipping 会累积，尤其影响 MoE RL 稳定性。它基于整条序列的似然构造 importance ratio，并做序列级 clipping、rewarding 和 optimization。

直觉上，rollout 的动作对象是整条回答，序列级 ratio 更贴合该采样单位，并减少 token 级异常比率对更新的破坏。代价是序列级信号对局部 token 的信用分配更粗，需要结合具体实现评估。

78. On-policy 与 Off-policy 在 LLM RL 中怎么区分？

On-policy 使用当前或非常接近当前策略生成的数据，分布匹配但数据一次性、rollout 贵；Off-policy 可复用历史轨迹或外部数据，样本效率高，但需重要性采样或其他校正，策略差距大时方差和偏差严重。

PPO/GRPO 通常近似 on-policy；DPO 是离线偏好学习，不同于传统意义的 off-policy policy gradient。异步 rollout 会产生 policy lag，需要版本控制、ratio 过滤和训练/推理权重同步。

第六章推理、量化、vLLM 与服务工程

推理面经通常围绕 prefill/decode、KV Cache、显存带宽、continuous batching、PagedAttention、量化和延迟吞吐权衡展开。

79. 大模型推理分为 Prefill 和 Decode 有什么意义？

Prefill 对完整输入 prompt 一次并行计算，主要是大矩阵乘，通常 compute-bound；Decode 每步只输入一个新 token，但要读取全部历史 KV Cache，矩阵形状小、内存访问占比高，通常 memory-bandwidth-bound。

因此两阶段优化不同：Prefill 关注 FlashAttention、chunked prefill、算力利用率；Decode 关注 KV Cache、continuous batching、GQA、量化、speculative decoding。在线指标也应分别看 TTFT 和 TPOT。

80. TTFT、TPOT、吞吐和并发分别是什么？

TTFT 是从请求到首 token 的时间，受排队、tokenize、prefill 和调度影响；TPOT 是首 token 后平均每个输出 token 的时间；端到端延迟约为 TTFT 加输出 token 数乘 TPOT。吞吐可按 request/s 或 token/s 衡量，并发是同时活跃请求数。

提高 batch 常提升吞吐但可能增加排队和单请求延迟。服务优化必须明确 SLA：交互式对话重视 TTFT/TPOT，离线批处理更重视总 token 吞吐和成本。

81. 为什么大模型推理时显存一直占着？

框架会缓存 CUDA 内存池以避免频繁 malloc/free，已释放张量的显存可能仍显示为 reserved；模型权重长期驻留；KV Cache 为活跃请求保留；算子 workspace、CUDA graph 和通信 buffer 也会常驻。

应区分 allocated、reserved、进程显存和实际可复用内存。empty_cache 只能将缓存池未使用块归还驱动，不会释放仍被引用的张量，也不应在每次请求后频繁调用。

82. vLLM 的 PagedAttention 解决了什么问题？

传统 KV Cache 为每个请求预分配连续最大长度空间，造成内部碎片和过度预留。PagedAttention 将 KV Cache 切成固定大小 block，用类似虚拟内存的 block table 映射逻辑 token 到物理块，按需分配并支持共享前缀、copy-on-write。

它提高 KV Cache 利用率和可服务 batch，从而提升吞吐。代价是需要支持非连续块访问的 attention kernel、页表管理和一定元数据开销。

83. Continuous Batching 是什么？

静态 batching 必须等待整批所有请求结束，短请求会被长请求拖住。Continuous/iteration-level batching 在每个 decode step 动态加入新请求、移除完成请求，使 GPU 持续保持高利用率。

调度器需要在 prefill 与 decode、长短请求、优先级和 SLA 之间平衡。过大的 batch 会提高吞吐但增加 TPOT；频繁大 prefill 还可能阻塞已有 decode，因此常使用 chunked prefill 或分离式部署。

84. FlashAttention 为什么更快、更省显存？

标准实现会把完整注意力分数和 softmax 中间量写入 HBM。FlashAttention 按块把 Q/K/V 搬到片上 SRAM，使用在线 softmax 逐块更新归一化统计和输出，避免物化 $n \times n$ 矩阵，并在反向重计算部分中间量。

它是精确注意力，不是近似稀疏化；理论 FLOPs 仍近似 $O(n^2d)$ ，但 HBM I/O 显著减少，显存对序列长度近似线性。实际速度取决于 GPU、head dimension、mask 和 kernel 支持。

85. INT8/INT4 量化为什么不一定同比加速？

量化降低权重存储和内存带宽，但速度取决于硬件是否有高效低精度 Tensor Core、反量化/fused kernel、矩阵形状和 batch。小 batch decode 常受带宽限制，权重量化收益明显；大 batch prefill 可能 compute-bound，若反量化开销高，速度不一定提升。

此外 KV Cache、激活和部分层可能仍为高精度。量化首先是容量优化，其次才是速度优化，必须实测 TTFT、TPOT 和吞吐。

86. PTQ 与 QAT 的区别是什么？

PTQ 在训练后用校准数据计算 scale/zero-point 并量化，成本低；QAT 在训练中模拟量化误差，让模型适应低比特，效果通常更好但训练复杂。LLM 权重量化常见 GPTQ、AWQ、SmoothQuant 等，分别利用二阶近似、激活感知或平滑权重/激活异常值。

选择应看权重/激活位宽、目标硬件 kernel、校准数据和可接受精度损失。只报“INT4”不足以说明实际格式和性能。

87. GPTQ 和 AWQ 的思路有何区别？

GPTQ 以逐层、逐列方式使用近似 Hessian 信息补偿量化误差，目标是最小化层输出重构误差；AWQ 观察少量 salient 权重通道对激活更重要，通过激活统计缩放保护这些权重，再做低比特量化。

二者多属于 weight-only PTQ。实际效果取决于模型、group size、校准集与推理 kernel，不应绝对化谁更好。

88. 什么是 Speculative Decoding？

用较小 draft model 快速提出多个候选 token，再由大 target model 一次并行验证。若候选被接受，可在一次 target 前向中推进多个 token；拒绝时按校正规则采样，保持与 target 原分布一致。

加速取决于 draft 速度、接受率、候选长度和验证开销。对易预测文本效果好，复杂推理或模型分布差距大时接受率低。变体还包括 Medusa、多 token prediction、self-speculative 等。

89. Prefix Cache 能优化什么？

若多个请求共享相同系统提示、文档前缀或多轮历史，可缓存对应 KV block，后续请求复用，减少 prefill 计算和 TTFT。Paged KV 管理使 block 级共享更方便。

需要对 token 序列、模型版本、adapter、位置编码和采样上下文做严格 cache key；隐私隔离与生命周期管理也很重要。它只优化共享前缀，不能复用不同内容的语义相似 prompt。

90. 为什么 decode 常是 memory-bound?

每生成一个 token，要从显存读取各层大量权重和历史 KV，但每个权重参与的计算量在小 batch 下有限，算术强度低，GPU 计算单元等待内存。提高 batch 可让权重在多个请求间复用，提高算术强度；量化则减少需要读取的字节。

Roofline 分析可以用 FLOPs/byte 判断瓶颈。不能把所有推理慢都归因于 attention，FFN 权重读取常同样关键。

91. 如何估算模型仅加载权重所需显存?

粗略为 参数量 × 每参数字节，例如 7B 模型 BF16 权重约 14GB，INT8 约 7GB，4-bit 理论约 3.5GB。实际还包括量化 scale/zero-point、embedding/lm_head 是否共享、对齐、框架 buffer 和并行副本。

MoE 要按总参数估算权重驻留，按激活参数估算每 token 计算。张量并行可近似把可切分权重除以卡数，但每卡仍有未切分参数、通信 buffer 和 KV Cache。

92. 在线大模型服务如何做容量规划?

先用真实 prompt/output 长度分布和目标 SLA 压测，分别测 TTFT、TPOT、P50/P95/P99、token/s、显存和失败率。根据并发、KV Cache 每 token 大小和最大上下文估算可容纳序列数，再考虑峰值流量、冗余、模型加载和滚动升级。

工程措施包括限流、排队、超时、取消、最大 token、模型路由、自动扩缩容、降级、cache、隔离长请求和 backpressure。平均 QPS 无法代表尾延迟风险。

第七章 RAG：从文档入库到检索、生成与评测

RAG 是牛客和小红书相关面经中最常见的应用方向。面试官通常从概念进入，随后追问 chunk、embedding、混合检索、rerank、动态更新、评测和线上故障排查。

93. 什么是 RAG? 它主要解决什么问题?

RAG 在生成前从外部知识库检索相关内容，将证据放入上下文，再由模型回答。它主要解决参数知识不及时、私有知识难注入、事实可追溯和频繁更新成本高的问题。

RAG 不能从根本上消除幻觉：检索可能漏召回或召回错误，生成器也可能忽略证据、错误归纳或引用错位。可靠系统必须同时优化知识库、检索、上下文构造、生成约束和评测。

94. 一个完整 RAG 系统的链路是什么?

离线侧：数据接入 → 解析/OCR → 清洗去重 → 结构化与元数据 → chunk → embedding → 建索引 → 版本与权限管理。在线侧：意图识别/路由 → query rewrite/decompose → 多路召回 → 融合与过滤 → rerank → 上下文压缩/拼接 → 生成 → 引用与事实校验 → 日志和反馈。

面试回答应结合项目给出每一步的具体技术、数据规模、延迟和指标，而不是只说“向量库检索后拼 prompt”。

95. Chunk 应该如何切分?

固定 token 窗口简单稳定；递归按标题、段落、句子切分可保留结构；语义切分根据 embedding 变化识别主题边界；表格、代码、FAQ 和合同应采用类型专用策略。常用 overlap 防止边界信息丢失。

chunk 太小缺上下文、召回碎片多；太大则 embedding 语义稀释、命中后引入噪声且占窗口。应按文档结构、问题粒度和模型窗口做实验，并评估 retrieval recall 与最终 answer quality，而非固定背 500 字。

96. 如何避免语义被 chunk 边界切断?

保留标题层级和父文档 ID，使用滑动 overlap；召回子块后扩展相邻块或 parent section；对跨段关系使用 parent-child indexing；表格保留表头和行列上下文；代码按函数/类/模块切分。

另一思路是先检索细粒度 chunk 以提高定位，再将其映射到较大父块供生成。这样兼顾召回精度与上下文完整性。

97. Embedding 模型和 Reranker 有什么区别?

Embedding 模型通常分别编码 query 与文档，离线预计算文档向量，可用 ANN 高效召回大量候选；Reranker 通常联合编码 query-document 对或做更深交互，精度高但成本大，只对几十到几百个候选重排。

这是 coarse-to-fine 结构：召回目标是高 recall，rerank 目标是高 precision。若 embedding 阶段没召回正确文档，reranker 无法补救。

98. 稠密检索和 BM25 各自优缺点是什么?

稠密向量擅长语义同义、自然语言改写和跨表达匹配，但对精确数字、型号、罕见实体、代码符号可能不稳；BM25 基于词项匹配，对专有名词和精确关键词强，但难处理同义改写。

企业 RAG 常采用 hybrid search，再用 RRF、加权分数或学习排序融合。分数尺度不同，不能不校准就直接相加。

99. RRF 是什么？为什么适合混合检索？

Reciprocal Rank Fusion 根据不同检索器中的排名融合：

$$\text{score}(d) = \sum_i \frac{1}{k + \text{rank}_i(d)}.$$

它不依赖不同检索器原始分数的可比性，对异常分数较稳健，适合 BM25 与向量召回合并。 k 控制头部排名影响。RRF 简单强健，但无法利用精确的置信度和业务特征，复杂场景可训练 learning-to-rank。

100. Query Rewrite 有哪些类型？

包括拼写与实体规范化、缩写展开、代词消解、对话历史改写为独立问题、生成多个视角的 multi-query、HyDE 生成假想答案向量、复杂问题拆成子问题，以及将自然语言转换成结构化过滤条件。

Rewrite 有可能改变原意或引入实体，应保留原 query 作为一路召回，并对高风险查询做约束。评测要区分 rewrite 后 recall 提升与误改写率。

101. Rerank 为什么常能显著提升效果？

ANN 召回为了速度采用双塔独立编码和近似搜索，候选中常混有语义相近但不能回答问题的片段。Cross-encoder reranker 可以对 query 与文档 token 做细粒度交互，识别否定、数字、条件和真正的可回答性。

通常先扩大 recall top-N，再 rerank 取 top-k。N 过小会漏召回，过大增加延迟；可采用轻量 reranker、批处理、early exit 或两级重排。

102. RAG 的上下文如何拼接？

先按相关性、来源权威性、时间、权限和去重过滤；可合并相邻块、保留标题和文档 ID；控制总 token 预算并为问题、指令和回答留空间。对冲实证据应同时呈现并标注时间/版本，而不是只拼一个最高分块。

可以使用 extractive compression 只保留与 query 相关句子，但压缩器可能丢掉限定条件。上下文顺序也影响模型注意，常将最关键证据放前后，并通过引用编号绑定。

103. 如何评估检索效果？

有标注相关文档时，常用 Recall@k、HitRate@k、Precision@k、MRR、MAP、NDCG。Recall@k 判断正确证据是否进入候选；MRR 关注首个相关结果排名；NDCG 适合多级相关性。

还应按查询类型分桶，例如实体、数字、多跳、时间敏感、无答案。离线指标必须与线上回答成功率和延迟联合看，高 recall 但噪声太多也会损害生成。

104. 如何评估 RAG 最终回答？

至少拆成：答案正确性/任务成功率；faithfulness，即结论是否由证据支持；context relevance；引用准确率和覆盖率；无答案识别；安全与权限；延迟和成本。

可用人工 rubric、LLM-as-a-Judge、事实抽取后逐项验证、RAGAS 类指标和端到端测试。Judge 必须做位置交换、盲评、与人工相关性校准；对关键业务应以可执行规则或人工金标为主。

105. RAG 召回效果不好时如何排查？

按层排查：数据是否解析丢失、乱码或版本旧；chunk 是否不合理；embedding 是否匹配语言和领域；索引和权限过滤是否正确；query rewrite 是否误改；ANN 参数和 top-k 是否过小；混合检索/融合是否失衡；reranker 是否把正确证据降权。

先建立可观测链路，保存每步候选和分数。用失败 query 做 error taxonomy，比盲目更换向量库更有效。

106. RAG 最大的落地瓶颈是什么？

常见瓶颈不是单一模型，而是知识质量与评测：文档持续变化、解析格式复杂、权限粒度、正确 chunk 缺失、标注成本高、跨文档推理、延迟与可解释性。线上用户 query 分布也会漂移。

成熟方案需要数据治理、版本化、增量索引、反馈闭环、分桶 benchmark 和故障降级。只优化 embedding benchmark 往往无法解决业务端到端问题。

107. 动态更新知识库应全量还是增量？

高频更新和大规模语料通常采用增量：为文档计算内容哈希和版本，识别新增/修改/删除，只重解析受影响块，使用 upsert 和 tombstone 更新索引。定期做全量重建校验以清理漂移和碎片。

需要确保 embedding 模型、chunk 策略和 schema 版本一致。若升级 embedding，旧向量与新向量不可直接混用，宜双索引迁移、后台重嵌入和灰度切换。

108. 如何避免新旧文档分布不一致导致检索偏差？

记录 embedding/version/source/time 元数据，必要时分索引；统一归一化和打分校准；对新旧数据构造联合评测；在融合阶段加入时间衰减、权威性和版本优先级；升级模型时做全量或双写迁移。

若只是新增业务领域，单一 embedding 可能产生域偏移，可做领域微调、增加 BM25 路径、query routing 或按领域使用不同 retriever。

109. 什么是 Parent-Child Retrieval？

将文档切成小 child chunk 用于 embedding 和精确召回，但命中后返回其较大的 parent section 给生成器。小块提高检索定位，大块保留上下文。

父子映射需要稳定 ID、去重和 token 预算控制。多个 child 命中同一 parent 时应合并，避免重复占用上下文。

110. Graph RAG 适合什么场景？

当问题依赖实体关系、跨文档多跳、全局主题总结或社区结构时，图谱可显式表示实体、关系和来源。查询可先做实体链接，再进行图遍历、子图检索或社区摘要。

代价是图构建与更新复杂、实体消歧困难、关系抽取有误差。FAQ 或简单文档问答不一定需要图数据库，不能为了“高级”而引入。

第八章 Agent、工具调用、MCP、记忆与多智能体

Agent 面经的核心不在框架 API，而在任务分解、状态管理、工具可靠性、反馈闭环、容错与评测。

111. Agent 的基本架构由哪些组件构成？

常见组件包括：LLM/策略核心；Planning 任务分解与下一步决策；Tools 与外部系统交互；Memory/State 保存上下文和长期信息；Observation 读取工具结果；Executor/Orchestrator 管理循环、权限、超时和终止；Evaluator/Guardrail 做验证与安全控制。

面试时不要只答“LLM+Prompt”。生产 Agent 的可靠性很大程度来自模型外的状态机、校验器、重试、权限与可观测性。

112. Agent 与 Workflow/Chain 有什么区别？

Workflow 的步骤和分支主要由开发者预定义，可预测、可测试、适合稳定流程；Agent 让模型根据观察动态选择工具和路径，适合开放、信息不完整的任务，但成本、延迟和不确定性更高。

实际系统常是混合架构：外层确定性 workflow 控制关键阶段，内部在局部使用 Agent 做搜索、规划或异常处理。能用确定性流程解决的环节不应全部 agentize。

113. ReAct 模式是什么？

ReAct 交替进行 reasoning、action 和 observation：模型根据目标决定下一步动作，调用工具获得观察，再更新计划，直到输出答案或达到终止条件。它将语言推理与外部交互结合，能根据真实结果纠错。

生产环境通常不会直接暴露完整 Thought，而是使用结构化 action/tool call 和内部状态摘要。必须设置最大步数、工具白名单、参数校验、循环检测和超时。

114. Function Calling 是如何训练和执行的？

模型在 SFT/偏好数据中学习：给定 system、用户请求和工具 schema，输出结构化工具名与参数；执行器校验 JSON/schema、权限和类型，调用真实函数，将结果作为 tool message 返回模型，再生成后续调用或最终回答。

模型只“提出调用”，不应直接拥有无限执行权限。可靠性来自 constrained decoding、schema validation、幂等设计、沙箱、重试和结果校验。

115. Function Calling 与普通 Prompt 输出 JSON 有何区别？

普通 Prompt 依靠自然语言约束，格式容易漂移；原生 function calling 通常有专门训练格式、结构化 token 或 API 协议，服务端可获得明确的 tool call 对象。两者底层仍可能是模型生成，但原生协议更便于校验、路由和多工具管理。

即使使用原生 FC，也不能相信参数语义一定正确，仍需业务校验和授权。

116. MCP 是什么？它与 Function Calling 有何关系？

MCP 是连接模型应用与外部 tools/resources/prompts 的标准化协议，定义客户端、服务器、能力发现和消息交互。Function Calling 主要描述模型如何选择并提供函数参数；MCP 解决工具如何被统一暴露、发现、调用和返回资源。

可以理解为：FC 是模型层的调用表达，MCP 是应用与工具生态的连接协议。使用 MCP 不会自动解决工具选择准确性、权限和安全问题。

117. 多步 Planning 有哪些实现方式？

可使用一次性 plan-and-execute、ReAct 边执行边规划、层次任务树、LLM 生成 DAG、状态机/图编排、MCTS/beam search, 或 planner 与 executor 分离。复杂任务常需要 replan: 工具失败、环境变化或中间假设被否定后更新剩余计划。

选择取决于任务可预测性和成本。一次性计划便宜但易受早期错误影响；逐步规划适应性强但调用多；显式 DAG 便于并行和可观测。

118. Agent 的短期记忆和长期记忆如何设计？

短期记忆包括当前对话、计划、工具结果和 scratch state, 受 context window 限制, 可用摘要、结构化状态和窗口裁剪。长期记忆存储跨会话偏好、事实或经验, 可用关系库、向量库或事件日志。

关键是写入门控、检索触发、时间衰减、来源、权限和可修改性。不能把所有对话无脑向量化, 否则会产生噪声、隐私风险和错误记忆。

119. Semantic、Episodic、Procedural Memory 分别是什么？

Semantic Memory 存事实、概念和稳定知识; Episodic Memory 存具体任务经历、尝试、结果和错误; Procedural Memory 存可复用流程、策略和技能。工程中可分别映射到知识库、轨迹日志和 workflow/skill 库。

检索时应按任务需要选择类型。例如 Debug Agent 需要先查历史相似失败 (episodic), 再查 API 规则 (semantic), 最后复用修复步骤 (procedural)。

120. Agent 如何避免无限循环？

设置最大 step/token/时间/费用预算; 对状态与动作序列做重复检测; 定义明确终止条件; 工具错误分类并限制重试次数; 使用 progress evaluator 判断是否产生新信息; 连续无进展时降级或请求人工。

循环通常源于工具返回不明确、模型未维护状态、目标不可达或错误重试策略。仅设置 max_steps 能止损, 但不能提高任务成功率。

121. 工具调用失败时如何设计 fallback？

先分类: 参数校验失败、权限不足、超时、限流、可重试服务错误、业务无结果、不可逆操作风险。对可重试错误做指数退避和抖动; 切换备用工具或模型; 缩小请求; 使用缓存; 回退到只读/人工确认; 最终返回透明的部分结果。

写操作必须幂等或使用 idempotency key, 避免超时重试导致重复支付、重复发信。Fallback 应由显式策略控制, 而不是让模型无限自由尝试。

122. 如何降低包含多个工具的 Agent 延迟？

先画出 critical path。可并行无依赖工具; 合并/批量调用; 缓存稳定结果; 减少重复规划; 用小模型做路由和参数抽取, 大模型处理复杂推理; 流式执行; 预取高概率工具; 对长工具设超时和降级。

还可将确定性步骤移出 LLM、缩短工具 schema 和历史上下文、使用并行 DAG。优化时要监控各节点 P50/P95、调用次数和成功率, 不能只看模型推理时间。

123. Single-Agent 与 Multi-Agent 如何选择？

Single-Agent 状态简单、成本低、易调试, 优先作为基线。Multi-Agent 适合角色确实需要不同工具/权限/上下文, 任务可并行, 或需要独立评审与对抗检查的场景。

多 Agent 会增加通信、上下文损失、协调失败、重复工作和成本。角色命名本身不会产生能力, 必须定义清晰协议、共享状态、终止与冲突解决机制, 并用 ablation 证明优于单 Agent。

124. 多智能体如何协同？

可采用 manager-worker、planner-executor-critic、黑板系统、消息队列、共享 DAG 或竞赛/投票。每个 Agent 的输入输出应结构化, 明确任务所有权、依赖、版本和置信度。

对冲突可以基于证据评分、独立 verifier、规则优先级或人工仲裁。共享全部对话会快速膨胀, 应传递任务摘要和必要 artifacts, 而非完整思维轨迹。

125. Reflection 机制是什么？为什么可能无效？

Reflection 让模型或独立 critic 分析上轮失败、总结原因并修改计划。它对工具错误、遗漏约束和可验证任务有帮助, 但同一个模型可能重复原偏差, 产生“看起来合理的自我批评”却不改变行为。

应给 reflection 真实 observation、测试结果和具体 rubric, 并要求生成可执行修改。最好用不同模型、规则验证器或环境反馈。反思次数也要受预算限制。

126. Agent 应如何评测？

至少包括任务成功率、步骤/工具选择正确率、参数正确率、规划效率、恢复率、幻觉与越权率、终止正确性、延迟、token/工具成本和稳定性。对长任务应评测中间里程碑，而不仅是最终文本相似度。

建立可回放环境和固定工具 mock，保留完整 trajectory；按错误类型分桶；对随机模型多次运行报告均值和方差。线上再看真实转化、人工接管和事故率。

127. 如何区分 Planning 能力与工具幻觉？

将评测拆开：给定完美工具描述，检查计划是否覆盖必要步骤和依赖；再给定固定计划，检查模型是否选择存在的工具、正确参数和使用 observation。也可用 oracle planner 或 oracle tool 形成交叉实验。

如果只看最终成功率，无法知道失败来自计划、检索、工具、环境还是执行器。模块化 ablation 是面试中很加分的回答。

128. Agent 面临哪些安全风险？

包括 prompt injection、间接注入、工具描述投毒、数据外泄、权限提升、危险写操作、供应链恶意 MCP server、跨租户记忆泄漏和自动化放大错误。

防御应采用最小权限、工具 allowlist、输入输出隔离、来源标记、敏感操作人工确认、沙箱、网络和文件访问控制、secret 不进入模型上下文、审计日志和策略引擎。不能只靠 system prompt。

第九章模型评测、幻觉、安全与 LLM-as-a-Judge

评测题往往用于判断候选人是否真正做过迭代。好的回答应能把模型能力、系统链路、业务指标和统计可信度分开。

129. 大模型应该如何评测？

采用分层体系：基础能力 benchmark 测知识、推理、代码和多语言；任务级离线金标测准确率/成功率；生成质量测帮助性、事实性、指令遵循和风格；安全评测覆盖越狱、偏见和隐私；系统评测覆盖延迟、吞吐、成本和稳定性；最后以在线 A/B 和业务指标验证。

任何单一分数都不够。必须固定模型版本、prompt、采样参数和评测集，报告置信区间，并对失败样本做 taxonomy。

130. Perplexity 是什么？有什么局限？

Perplexity 是平均 token 负对数似然的指数：

$$\text{PPL} = \exp \left(-\frac{1}{N} \sum_{t=1}^N \log p(x_t | x_{<t}) \right),$$

越低表示模型对该语料分布预测更好。

它受 tokenizer 影响，不同词表模型不可直接比较；低 PPL 不保证指令遵循、事实性或推理能力；数据污染会让数值虚低。它适合监控预训练/继续训练和同 tokenizer 的模型对比。

131. Pass@k 如何计算和理解？

代码或数学任务对每题采样 n 个答案，其中 c 个正确，在不放回抽取 k 个时至少一个正确的无偏估计常写为：

$$\text{pass}@k = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}.$$

Pass@1 反映单次可靠性，Pass@k 反映通过多采样与验证获得正确解的潜力。必须统一 temperature、 n 、验证器和去重策略，不能把 best-of- k 与单次线上效果混为一谈。

132. LLM-as-a-Judge 有哪些偏差？

常见位置偏差、长度偏差、措辞/风格偏差、自家模型偏好、参考答案锚定、对事实无法验证，以及对 prompt injection 的脆弱性。Judge 也可能给出高一致但错误的理由。

缓解包括交换候选顺序、多次评审、多 Judge ensemble、隐藏模型身份、长度归一化、明确 rubric、要求引用证据、与人工标注做校准，并对关键指标使用可执行验证器。

133. 如何保证评测统计可信？

评测集应覆盖真实分布与关键长尾，样本量足够；固定随机种子并多次采样；报告均值、方差、bootstrap 置信区间或显著性检验；避免在同一测试集反复调参导致过拟合。

线上 A/B 要控制流量分层、时段和用户差异，并预先定义主指标、护栏指标和停止规则。只展示“几个好例子”不构成有效评测。

134. 大模型幻觉有哪些类型？

可分为内在幻觉，即输出与给定上下文矛盾；外在幻觉，即生成无法由上下文或事实支持的内容；还包括伪造引用、错误工具结果、推理链中间错误和过度自信。

原因包括训练目标是流畅预测而非事实验证、知识过时、检索噪声、解码随机性、提示歧义和奖励偏差。应按类型定位，而不是笼统说“接 RAG 就解决”。

135. 如何降低幻觉？

组合措施包括：高质量 SFT/偏好数据；RAG 与可靠引用；工具或数据库查询；明确允许拒答；结构化输出与约束解码；生成后事实核验；多样本自一致性；不确定性/置信度校准；对高风险任务人工复核。

关键是让系统在不知道时能识别和停止。要求模型“绝对不要幻觉”的 prompt 作用有限，必须提供外部证据和验证闭环。

136. 如何让模型输出稳定、一致且格式合规？

降低 temperature、固定 seed（但跨硬件/版本未必完全可复现）、使用明确 schema 和示例；采用 JSON schema constrained decoding/grammar；对输出做 parser 校验、自动修复和重试；关键字段由规则或工具生成。

语义一致性不能仅靠低温度，还需稳定上下文、模型版本、检索结果和工具结果。线上应记录 prompt/template/model/index 版本以可复现。

137. Prompt Injection 与 Jailbreak 有何区别？

Jailbreak 通常是用户直接构造输入绕过安全规则；Prompt Injection 更广，包括文档、网页、邮件或工具结果中的间接恶意指令，诱导 Agent 忽略原目标、泄露数据或调用危险工具。

二者都利用模型难以把“数据”和“指令”严格隔离。防御依赖权限与隔离架构，而不是只加更长 system prompt。

138. 大模型安全系统应采用哪些防线？

输入侧分类和敏感数据处理；模型侧安全 SFT/偏好对齐；检索与工具侧最小权限、内容来源和沙箱；输出侧策略检测、PII/秘密扫描、事实与引用验证；系统侧限流、审计、红队和事件响应。

应假设单层会失败，采用 defense-in-depth。对 Agent 写操作尤其需要人机确认、事务边界和可撤销性。

第十章多模态大模型基础

多模态岗位除了通用 LLM 八股，通常还会追问视觉特征如何接入、对齐数据、评测和推理成本。

139. 多模态大模型常见结构是什么？

典型结构是视觉编码器（如 ViT）提取图像 patch 特征，通过 projector/Q-Former/resampler 映射到 LLM 隐藏空间，再与文本 token 拼接或通过 cross-attention 交互。训练可分为视觉-语言对齐预训练、指令微调 and 偏好/强化学习。

不同模型可能冻结视觉编码器和 LLM，只训练连接层；也可能端到端更新。回答应明确输入分辨率、视觉 token 数和位置处理。

140. 图像特征是如何“喂给”大模型的？

图像切成 patch，经视觉 Transformer 得到一系列视觉 token；projector 将其维度映射到 LLM embedding 维。随后可把视觉 token 插到特殊 <image> 位置，与文本统一做 causal self-attention；或让文本 Query 通过 cross-attention 读取视觉特征。

视觉 token 数直接影响上下文长度和计算。高分辨率常用 dynamic tiling、AnyRes、token pruning 或 resampler 压缩。

141. CLIP 的训练目标是什么？

CLIP 用批内图文对进行对比学习，将匹配图像和文本向量拉近，非匹配对推远。通常计算归一化 embedding 的相似度矩阵，并分别做 image-to-text 和 text-to-image 交叉熵。

CLIP 擅长全局语义对齐和零样本分类，但细粒度空间关系、计数和 OCR 可能不足。多模态生成模型通常需要更细粒度视觉编码与指令数据。

142. 多模态对齐的主要难点是什么？

包括视觉与语言粒度不一致；图文数据噪声；空间关系和 OCR；高分辨率带来的 token 成本；语言先验压过视觉证据；多图/视频时序；幻觉和对象不存在。

应设计 hard negative、区域级/grounding 数据、OCR 与图表数据、视觉问答和拒答样本，并用图文一致性、grounding、OCR、计数与人工评测分维度验证。

143. 如何评估图文生成或理解质量？

理解任务可用 VQA accuracy、OCR/ChartQA、grounding IoU、caption 指标和人工 rubric；生成图像可评估文本-图像一致性、美学、真实感、主体身份、文字渲染和安全。CLIPScore/FID 等自动指标只反映部分维度。

业务中更应构建场景化测试集和 pairwise 人评，特别关注失败率、敏感内容、中文文字和长尾组合。

144. 多模态模型为什么更容易出现视觉幻觉？

语言模型拥有强先验，视觉特征弱或被压缩时，模型会按常识补全不存在的对象；训练数据中 caption 也可能不精确。长文本指令、低分辨率、小目标和多图混合会进一步稀释视觉证据。

缓解方法包括更强视觉编码、区域级对齐、grounding supervision、对比负样本、视觉 token 保留、生成后检测器验证，以及明确允许“图中看不清/不存在”。

第十一章项目拷打与系统设计题

平台面经显示，面试官常用同一组问题验证项目是否真实：为什么做、数据从哪来、基线是什么、指标如何提升、失败样本是什么、重新设计会怎样。

145. 如何用三分钟讲清一个大模型项目？

采用“背景-目标-方案-个人贡献-结果-反思”结构。先用一句话说明业务痛点和量化目标；再画出核心链路；明确你负责的模块和关键决策；给出基线、数据规模、离线/在线指标、延迟成本；最后讲一个失败案例和下一步。

避免只罗列框架名称。面试官更关注为什么选择该方案、如何证明有效、你亲自解决了什么。

146. 为什么选择 RAG，而不是微调？

若目标是频繁更新、私有事实、可引用和低训练成本，RAG 更合适；若目标是稳定风格、格式、隐式决策模式或减少每次检索延迟，微调更合适。两者可组合：SFT 学会领域任务与引用行为，RAG 提供实时事实。

应结合数据变化频率、知识量、权限、延迟、评测和维护成本回答，而不是说“RAG 不改参数、微调会改参数”就结束。

147. 项目效果提升应如何做消融？

固定数据与评测集，逐项增加组件：基础 prompt → query rewrite → hybrid recall → rerank → context compression → fine-tune。记录每一步对 recall、faithfulness、成功率、延迟和成本的影响。

对于相互作用强的组件可做 factorial ablation。线上还要做灰度 A/B。没有消融时，很难说明提升来自哪一项，也容易被面试官质疑“只是换了更强模型”。

148. 如何分析 Bad Case？

先建立错误 taxonomy，例如知识缺失、检索漏召回、排序错误、上下文冲突、指令误解、工具参数、生成幻觉、格式失败和评测器误判。保存端到端轨迹，定位最早发生错误的节点。

对每类统计占比和业务损失，优先修高频高风险类别；为已修问题加入回归集。优秀回答应给出具体 bad case、根因、修复和指标变化。

149. 如果模型效果比对标模型低 6%，你怎么处理？

先确认指标方差与显著性，保证 prompt、采样、数据和评测公平；按任务/长度/难度/语言分桶找到差距来源；做 oracle 实验判断瓶颈在检索、模型还是工具；比较错误类型和成本。

然后针对性优化数据、训练、检索或推理，并考虑是否值得用更高成本换 6%。业务决策看收益、延迟和价格的 Pareto，不是只追逐单一准确率。

150. 如何设计企业级大模型应用的降级策略？

按层降级：高端模型超时切换轻量模型；Agent 失败退化为固定 workflow；RAG 检索失败只返回可验证已知信息；工具不可用时使用缓存或只读模式；高风险写操作转人工；超长请求做摘要或拒绝。

需要熔断、限流、超时、重试上限、健康检查和可观测性。降级结果必须透明，不能悄悄编造。

151. 如何进行模型选型？

先定义任务、语言、上下文、工具、多模态、安全、部署和预算约束；建立代表性评测集；比较质量、TTFT/TPOT、吞吐、显存、价格、许可、数据合规和生态；对候选做小规模 PoC 与压力测试。

模型榜单只是筛选。真实 prompt 分布、量化版本和 serving framework 会改变结果。应保留多模型路由和替换

能力，避免被单一供应商锁定。

152. 如果重新设计你的 Agent/RAG 项目，会改什么？

先基于现有 bad case 说出最重要的结构性问题，而不是泛泛说“换更强模型”。例如把自由 Agent 改成 workflow+ 局部 Agent；加入可回放环境；将单路向量检索改为 hybrid+rerank；为动态文档加版本化；把 LLM judge 与规则验证结合。

面试官借此判断你是否有迭代意识。回答应包含优先级、预期收益、风险和验证方法。

第十二章常见手撕题、计算题与快速追问

大模型算法岗仍会考 PyTorch、复杂度、显存与传统算法。以下给出可口述的实现要点。

153. 如何用 PyTorch 手写 Multi-Head Attention？

关键步骤：线性映射得到 $[B, T, H, D]$ 的 $Q/K/V$ ；转置为 $[B, H, T, D]$ ；计算 $\text{scores} = Q @ K.transpose(-2, -1) / \sqrt{D}$ ；应用 causal/padding mask；softmax 和 dropout；与 V 相乘；转回 $[B, T, H * D]$ 并输出投影。

要注意 mask 广播形状、softmax 维度、连续内存、cross-attention 时 Q 与 K/V 长度不同。实际面试代码应避免显式 Python 循环。

154. Softmax 如何实现数值稳定？

先减去每行最大值 $m = \max_i x_i$ ，再计算

$$\text{softmax}(x)_i = \frac{e^{x_i - m}}{\sum_j e^{x_j - m}}.$$

减常数不改变 softmax 结果，却避免大正数指数溢出。

交叉熵实现通常使用 log-sum-exp 融合，而不是先 softmax 再 log。低精度训练中某些归一化会临时升到 FP32。

155. 如何估算 Attention 的 FLOPs？

$Q/K/V$ 与输出投影大约为 $4nd^2$ ；注意力 QK^T 和 AV 各约为 $2n^2d$ （若乘加算 2 FLOPs，常数约定会不同），总量近似 $O(nd^2 + n^2d)$ 。

面试时先声明 FLOPs 计数约定，再说明短序列由线性层主导、长序列由 n^2 项主导。

156. 如何估算 LoRA 参数量？

对一个 $d_{out} \times d_{in}$ 权重，LoRA 的 A/B 参数量为 $r(d_{in} + d_{out})$ ，而全量参数为 $d_{in}d_{out}$ 。若覆盖多个投影层则求和，再乘层数。

例如方阵 $d \times d$ 的一个投影，参数比例约为 $2r/d$ 。但训练显存还包括 LoRA 梯度与优化器状态，以及冻结基座的激活。

157. Top-k、Top-p、Temperature 分别怎么影响生成？

Temperature 将 logits 除以 T ： $T < 1$ 更尖锐确定， $T > 1$ 更平坦随机。Top-k 只保留概率最高 k 个 token；Top-p 保留累计概率达到 p 的最小集合，候选数随分布变化。

通常先 temperature 再截断并重新归一化。推理/代码任务常低温；创作任务提高多样性。不能通过采样参数修复模型事实知识不足。

158. Beam Search 为什么不常用于开放式对话？

Beam Search 优化高概率序列，容易产生通用、重复和长度偏置的回答，且需要维护多个 beam，增加 KV Cache 与计算。开放式生成存在多个合理答案，采样更自然。

翻译、语音识别或结构化任务中 beam 仍有价值。可配合长度惩罚、重复惩罚和约束解码。

159. 为什么训练 loss 正常但生成全是重复内容？

可能原因包括数据模板重复、EOS 处理错误、label shift/mask bug、学习率过大导致退化、解码 repetition penalty/temperature 配置、KV Cache 或 position id 错误、过拟合、RL 熵坍塌。

排查顺序是用 teacher-forcing 检查 logits 和小样本过拟合；对比 greedy 与 sampling；验证 tokenizer/EOS/template；检查训练前后固定 prompt；查看 token entropy 和重复率。

160. 面试中不会某个前沿算法时怎么回答？

先明确边界：“我没有实际训练过，但理解它要解决的问题和与基线的差异。”随后从问题、核心机制、代价、适用场景和验证方法展开，避免编造公式或实验结论。

例如不熟 GSPO，可先准确说明 GRPO 的 token-level ratio 和长序列稳定性问题，再说 GSPO 改用序列级 ratio；最后指出仍需阅读实现确认细节。这比自信地说错更可靠。

附录 A：高频项目追问清单

以下问题几乎可套到任何简历项目，建议逐项写出自己的量化答案：

- ☐ 项目为什么立项？原方案最痛的指标是什么？
- ☐ 你负责的边界是什么？哪些代码或决策由你完成？
- ☐ 数据从哪里来？规模、清洗、去重、划分和合规怎么做？
- ☐ 为何选这个基座模型、embedding、reranker、Agent 框架或训练算法？
- ☐ 最初 baseline 是什么？每次迭代带来多少收益？
- ☐ 离线指标与线上业务指标分别是什么？是否显著？
- ☐ 一个完整请求的链路、耗时分解和成本是多少？
- ☐ 最大的 bad case 是什么？最早在哪个节点出错？
- ☐ 发生超时、工具失败、检索为空、模型格式错误时如何降级？
- ☐ 如何保证数据/索引/模型/Prompt 版本可追踪和可回滚？
- ☐ 如何防止评测泄漏、LLM Judge 偏差和只挑好样例？
- ☐ 重新设计时，第一优先级修改什么？为什么？

一分钟项目回答模板

“这个项目解决的是【业务问题】，原基线在【指标】上只有【数值】。我负责【个人模块】。整体方案是【3-5 个关键步骤】，关键决策是【为什么选该方法】。我们用【数据规模/评测集】做评测，最终【核心指标】从【A】提升到【B】，同时延迟/成本为【数值】。最大的失败类型是【bad case】，根因是【根因】，通过【修复】后下降了【数值】。如果重做，我会优先【下一步】。”

附录 B：资料来源

面经与题目来源

- **[S1]** 牛客：2024 | 大模型算法工程师相关面试题汇总：<https://www.nowcoder.com/discuss/645706098464301056>
- **[S2]** 牛客：大模型算法面经 + 问题 + 答案：<https://www.nowcoder.com/discuss/891322059656052736>
- **[S3]** 牛客：腾讯/百度/Minimax 大模型算法面经总结：<https://www.nowcoder.com/feed/main/detail/8cab51ef9458459595cf81fda43da279>
- **[S4]** 牛客：Minimax 大模型算法面经：<https://www.nowcoder.com/feed/main/detail/0380bd3ab1ba40f48865ee>
- **[S5]** 牛客：字节跳动大模型算法实习面经：<https://www.nowcoder.com/feed/main/detail/41e59b1a6f734d63ab81d9>
- **[S6]** 牛客：26 大模型算法校招面经（阿里）：<https://www.nowcoder.com/feed/main/detail/c261338c93de4ea5b3f96432baef2876>
- **[S7]** 牛客：大模型应用开发面经（5 年经验）：<https://www.nowcoder.com/feed/main/detail/129eaa1c20444651ac3b932e200d3da4>
- **[S8]** 牛客：货拉拉大模型算法一面面经：<https://www.nowcoder.com/feed/main/detail/30f49a12225a40f98d5ee763>
- **[S9]** 牛客：小红书大模型算法面经：<https://www.nowcoder.com/feed/main/detail/32594855133d427ab983700e23>
- **[S10]** 牛客：得物大模型面经八股：<https://www.nowcoder.com/feed/main/detail/b9be612ff28b4d9daba031552fa>
- **[S11]** 公开索引/二次整理：26 秋招小红书大模型技术岗位面试题：https://blog.csdn.net/2401_83173765/article/details/148292651
- **[S12]** 公开索引/二次整理：小红书多模态算法面试题：<https://blog.csdn.net/amusi1994/article/details/142581850>

答案校正与进一步阅读（原始论文）

- **[P1] Attention Is All You Need**: <https://arxiv.org/abs/1706.03762>
- **[P2] LoRA: Low-Rank Adaptation of Large Language Models**: <https://arxiv.org/abs/2106.09685>
- **[P3] QLoRA: Efficient Finetuning of Quantized LLMs**: <https://arxiv.org/abs/2305.14314>
- **[P4] Direct Preference Optimization**: <https://arxiv.org/abs/2305.18290>
- **[P5] DeepSeekMath / GRPO**: <https://arxiv.org/abs/2402.03300>
- **[P6] DeepSeek-R1**: <https://arxiv.org/abs/2501.12948>
- **[P7] DAPO**: <https://arxiv.org/abs/2503.14476>

- **[P8] GSPO**: <https://arxiv.org/abs/2507.18071>
- **[P9] ZeRO**: <https://arxiv.org/abs/1910.02054>
- **[P10] FlashAttention**: <https://arxiv.org/abs/2205.14135>
- **[P11] vLLM / PagedAttention**: <https://arxiv.org/abs/2309.06180>

免责声明

面经帖子可能存在记忆偏差、转述误差、付费截断或时间变化；本文对题目做了归类与改写，答案以公开论文和通用工程实践为准。具体公司、团队和岗位会根据业务方向继续深挖推荐、搜索、多模态、训练系统或安全等专项。